

PROCESADORES DE LENGUAJE - trad3

Grupo 206

11/03/2025

Jorge Mejías Donoso 100495807@alumnos.uc3m.es

Alberto Menchen Montero 100495692@alumnos.uc3m.es

1. Introducción

El presente documento describe el desarrollo de un traductor que convierte un subconjunto del lenguaje C a una notación intermedia en Lisp (prefijo) y posteriormente a notación Postfix. Esta tercera parte de la práctica final se centra tanto en completar el frontend como en desarrollar el backend que transforma el código Lisp intermedio en código final ejecutable sobre una máquina de pila.

2. Objetivos

- Ampliar el traductor para cubrir más elementos del lenguaje C, como estructuras de control, funciones y vectores.
- Asegurar una correcta conversión de código C a Lisp utilizando una representación prefija.
- Implementar un sistema de traducción a notación Postfix a partir del código Lisp generado.
- Validar el traductor usando programas de prueba reales como `prueba.txt`.

3. Estructura del traductor

El archivo principal del traductor es `trad3.y`, desarrollado con **Bison** y complementado por **Flex**. La estructura implementada es modular y jerárquica, siguiendo las recomendaciones del enunciado para evitar ambigüedades y problemas de recursividad innecesaria.

El traductor está dividido en:

- **Análisis léxico:** detección de tokens como identificadores, números, operadores, cadenas, símbolos y palabras reservadas.
- **Análisis sintáctico:** gramática jerárquica para traducir sentencias de C a expresiones Lisp, respetando precedencias.
- **Traducción diferida:** se usa generación intermedia de cadenas (`gen_code`) y estructuras de tipo árbol para construir la salida Lisp.

4. Elementos soportados

El traductor implementa correctamente los siguientes elementos:

4.1 Variables

- **Globales:** `int a = 2, b;` se traduce como `(setq a 2) (setq b 0)`
- **Locales:** definidas dentro del cuerpo de funciones con nombres decorados (`main_a`, `is_even_ep`, etc.)

4.2 Funciones

- Funciones sin parámetros y con parámetros, con retorno (estructura final en Lisp: `(defun name (args) ...)`)
- Se utiliza `return-from` cuando el `return` no es la última instrucción.

4.3 Impresión

- `puts("Hola")` → `(print "Hola")`
- `printf("%d", expr)` → `(princ expr)` — la cadena de formato no se traduce.

4.4 Operadores

- Aritméticos: `+`, `-`, `*`, `/`, `%`
- Comparación: `==`, `!=`, `<`, `<=`, `>`, `>=`
- Lógicos: `&&`, `||`, `!`

4.5 Estructuras de Control

- `if, if-else` → `(if cond cuerpo1 cuerpo2)` o con `progn` si hay múltiples instrucciones.
- `while` → `(loop while cond do cuerpo)`
- `for` → transformado internamente a `while` añadiendo la inicialización e incremento.

4.6 Vectores

- Declaración: `int v[10];` → `(setq v (make-array 10))`
- Lectura: `v[i]` → `(aref v i)`

- Escritura: `v[i] = x;` $\rightarrow$ `(setf (aref v i) x)`

5. Backend: LISP a Postfix

En la fase backend, el código Lisp generado se procesa para obtener una representación en notación postfix. Para ello, se ha implementado un recorrido del árbol sintáctico que invierte la notación prefija y aplica reglas específicas para operadores, condicionales y estructuras de bucle.

Ejemplo: `(+ 3 (* 2 5))` $\rightarrow$ `3 2 5 * +`

Este proceso asegura compatibilidad con una máquina de pila simple.

6. Pruebas realizadas

Se utilizó el archivo `prueba.txt` con las siguientes funciones:

- `square(int v)` $\rightarrow$ cuadrado de un número
- `fact(int n)` $\rightarrow$ cálculo recursivo del factorial
- `is_even(int v)` $\rightarrow$ imprime si un número es par o impar

Todas las funciones se tradujeron correctamente a Lisp, incluyendo sentencias internas con `if`, `return`, y `printf`. El traductor genera la salida esperada y puede ejecutarse con:

```
cat prueba.txt | ./trad3
```

8. Trabajo realizado durante la semana

Durante las sesiones de esta semana, nos hemos centrado principalmente en completar y pulir el traductor `trad3.y` para cumplir con todos los requisitos del enunciado. Las tareas realizadas incluyen:

- **Depuración final del frontend:** revisamos y ajustamos la gramática para asegurar una correcta traducción de todas las estructuras, especialmente `if-else`, `while`, `for` y funciones con múltiples parámetros y `return`.

- **Implementación y prueba de vectores:** se añadió el soporte completo para declarar, acceder y modificar vectores en C, traducidos adecuadamente a `make-array`, `aref` y `setf` en Lisp.
- **Revisión de la gestión de variables locales:** se mejoró el sistema de renombrado interno de variables locales, asociándolas correctamente al nombre de la función para evitar colisiones con variables globales.
- **Validación con `prueba.txt`:** se utilizó un archivo de pruebas con varias funciones (`square`, `fact`, `is_even`) para comprobar que la salida generada en Lisp era correcta y funcional bajo el intérprete `clisp`.
- **Preparación del backend:** se inició la implementación de la conversión de Lisp a Postfix, incluyendo el diseño del recorrido en profundidad del árbol generado para transformar correctamente las expresiones prefijas.
- **Documentación y formato:** se dio formato legible al archivo `trad3.y`, añadiendo comentarios y asegurando que todo el código fuera comprensible y mantenible, tal como se solicita en el enunciado.

9. Conclusión

Se ha implementado con éxito un traductor que cubre un amplio subconjunto del lenguaje C, respetando la estructura y sintaxis esperadas. Se han traducido correctamente estructuras de control, impresión, funciones y vectores, cumpliendo con los requisitos del enunciado. El backend complementa la solución con una notación postfix funcional, lista para su ejecución en una máquina de pila.